



UNIVERSIDAD NACIONAL MAYOR DE SAN MARCOS

Universidad decana de América

Facultad de Ingeniería de Sistemas e Informática

Escuela Profesional Ingeniería de Sistemas.

Estructura de Datos

Prof. Gilberto A. Salinas

Semana 10

Estructura de datos no lineales

Arboles Binarios

Arboles Binarios de Busqueda

Ciclo 2026-1

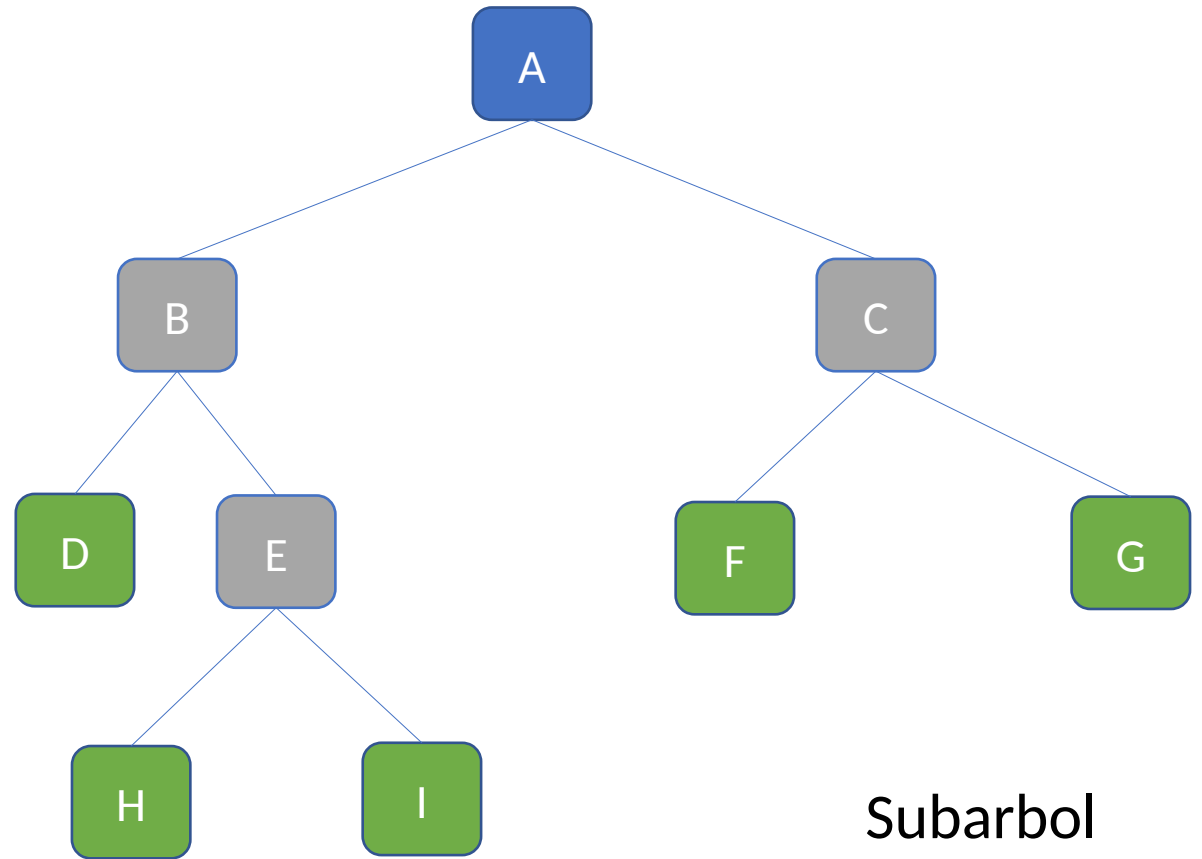
Agenda

1. Árboles binarios
 - Definición
 - Implementación
 - Operaciones
2. Árboles binarios de búsqueda (ABB)
 - Introducción
3. Definición
 - Características
4. Ventajas y desventajas
5. Implementación/Operaciones
6. Aplicaciones
7. Resumen
8. Referencias

Arboles Binarios

1. Definición.

- Arbol completo
 - Si todo nodo interno (no hoja) tiene dos hijos.



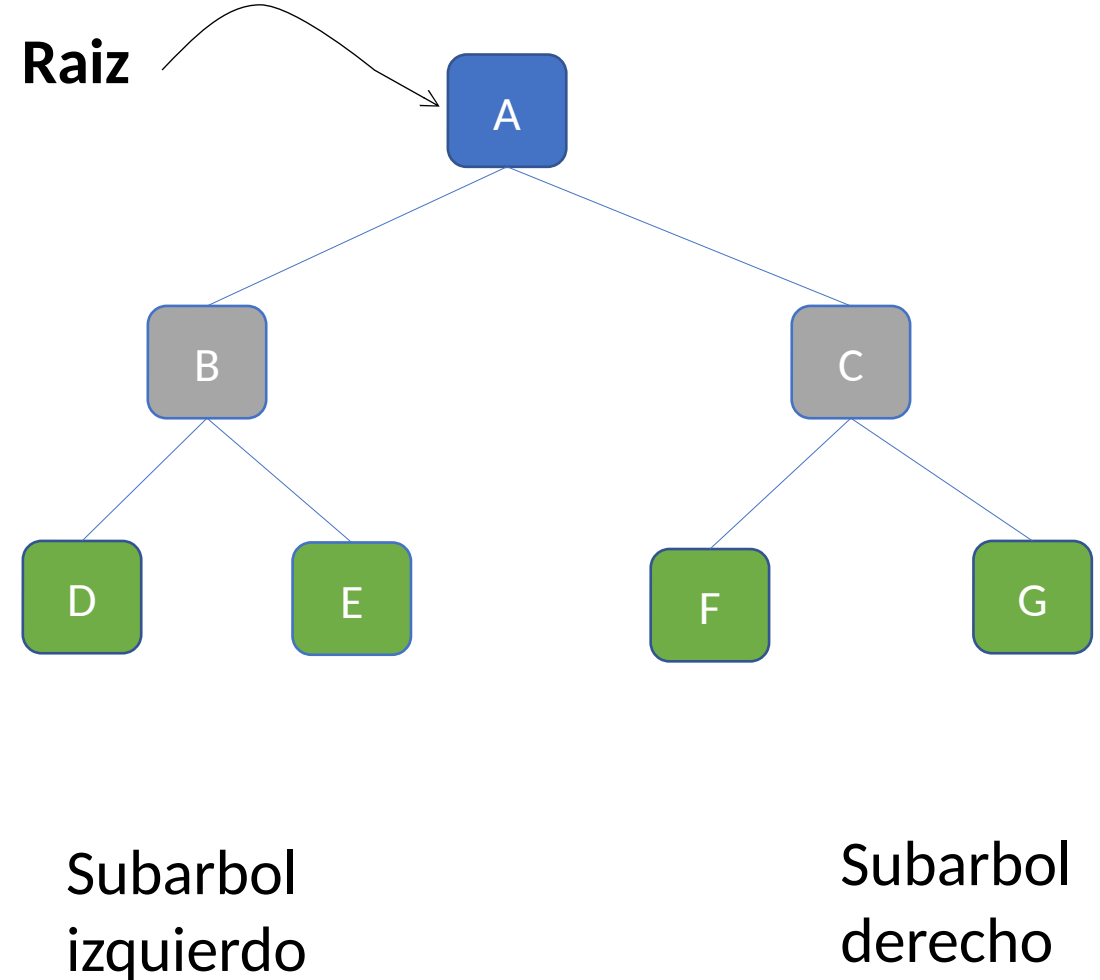
Subarbol
izquierdo

Subarbol
derecho

Arboles Binarios

1. Definición

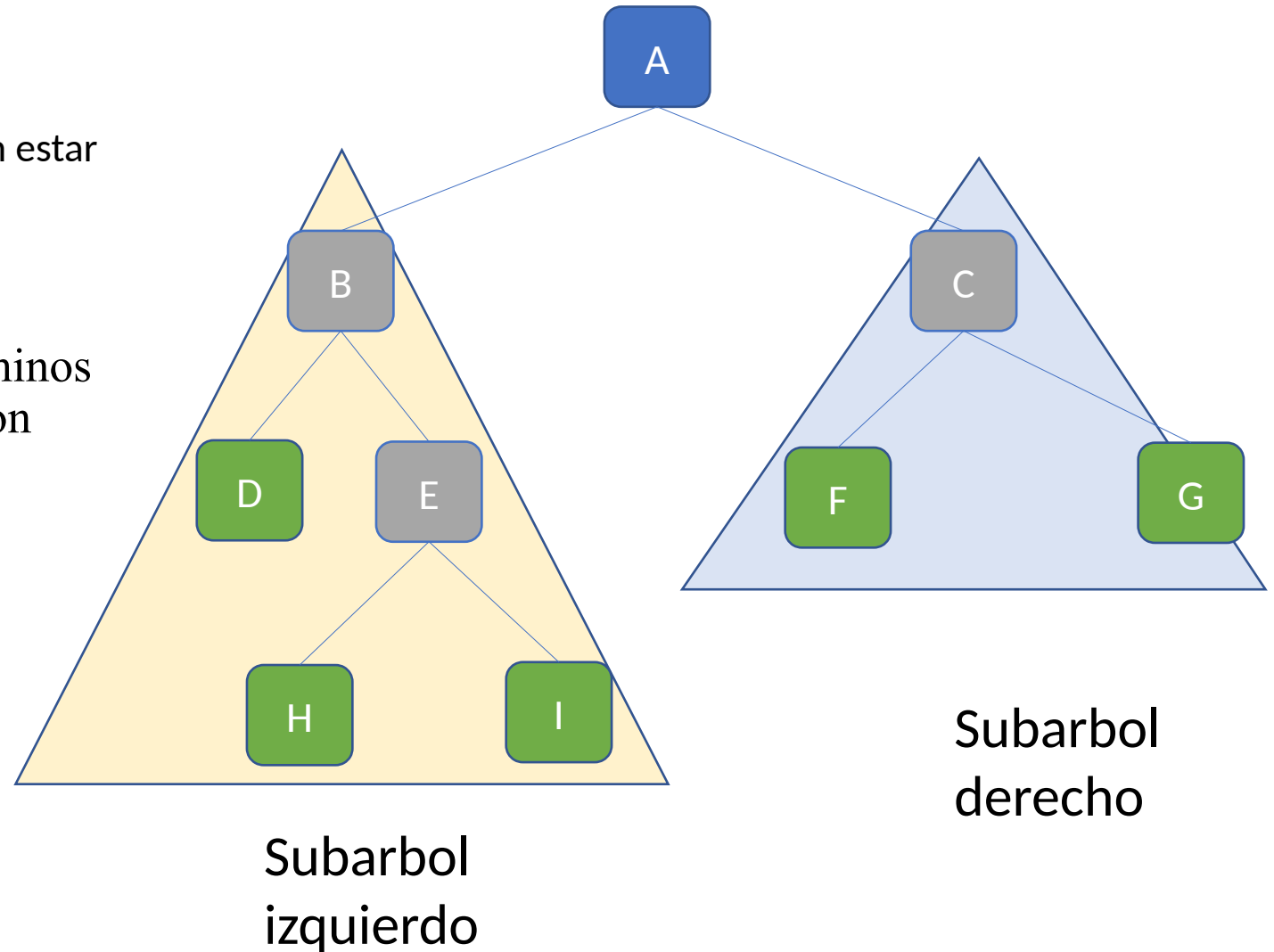
- Arbol lleno
 - Si es un arbol completo y todas sus hojas estan en el mismo nivel.



Arboles Binarios

1. Definición recursiva

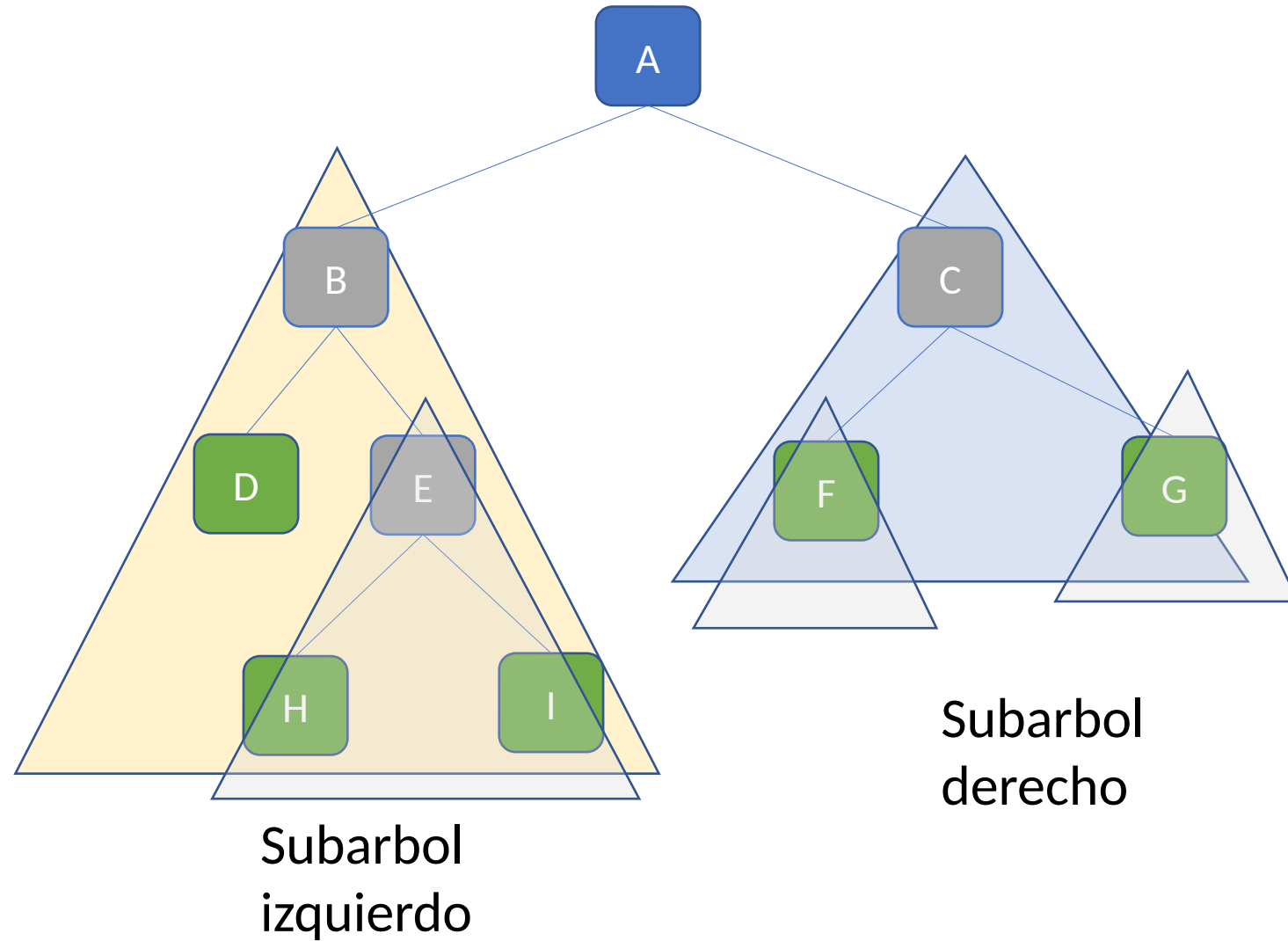
- Arbol de grado 2
 - Cada nodo tiene dos sub arboles (pueden estar vacíos)
 - Nodos internos: ABCE
 - Nodos externos u hoja: DFGHI
- **Estructura recursiva.** Se define en términos de sí misma o que contiene partes que son estructuralmente idénticas al todo.



Arboles Binarios

1. Definición recursiva

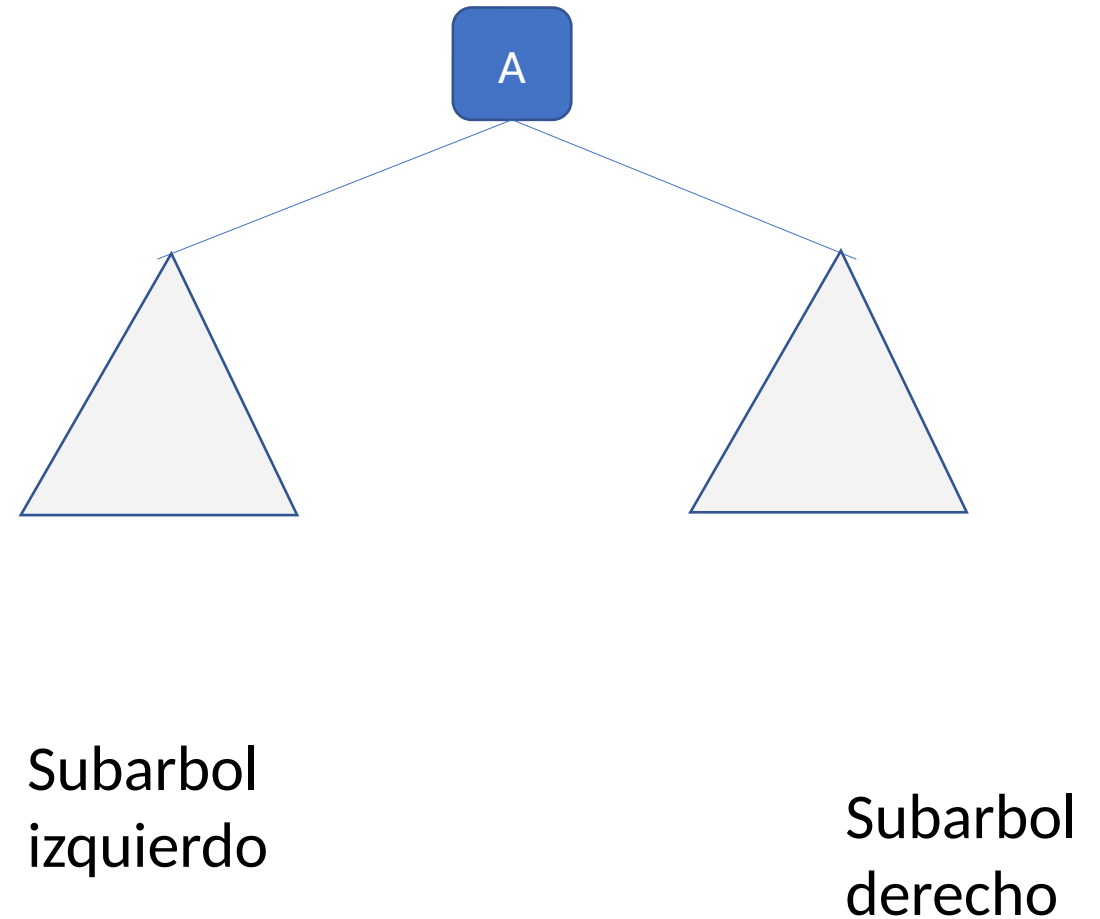
- Arbol de grado 2
 - Cada nodo tiene dos sub arboles (pueden estar vacíos)
 - Nodos internos: ABCE
 - Nodos externos u hoja: DFGHI



Arboles Binarios

1. Definición recursiva

- Arbol de grado 2
 - Cada nodo tiene dos sub arboles (pueden estar vacios)
 - Nodos internos: ABCE
 - Nodos externos u hoja: DFGHI



2. Árbol Representación dinamica

El árbol binario esta formado de nodos

- Cada nodo consta de tres campos o partes:
 - Información: data o información del árbol
 - Apuntador al hijo izquierdo
 - Apuntador al hijo derecho

```
REGISTRO NODO
    NODO *hi
    TD info
    NODO *hd
FIN_REGISTRO
```



3. Árbol. Representación estática

Representación en un arreglo: para arboles completos:

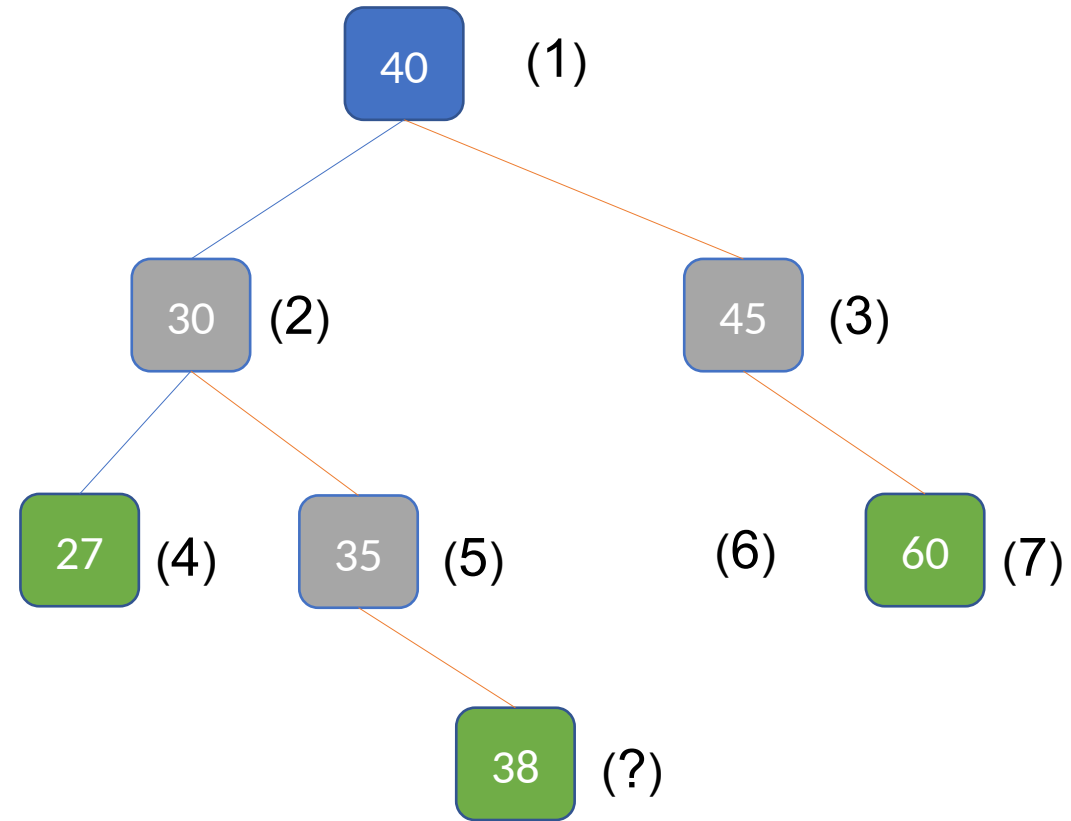
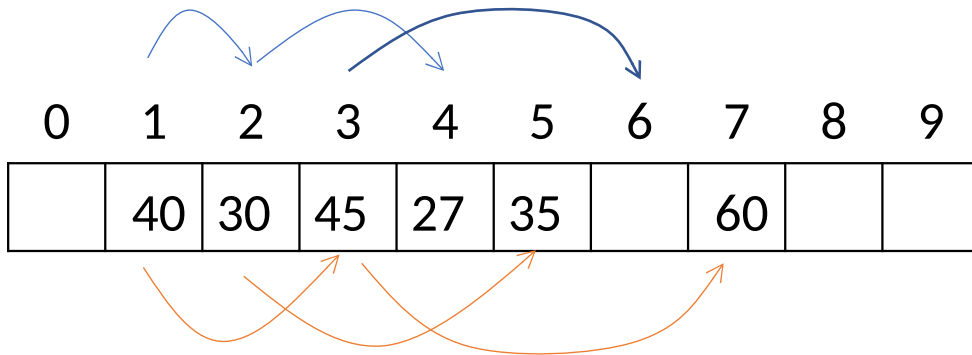
$$p(\text{raiz}) = 1$$

$$p(i.\text{left}) = 2 * p(i)$$

$$p(i.\text{right}) = 2 * p(i) + 1$$

HI: posicion $i = 2k$, donde $k = 1$ es para la raiz

HD: posicion $i = 2K + 1$



2. Implementación

Operaciones básicas:

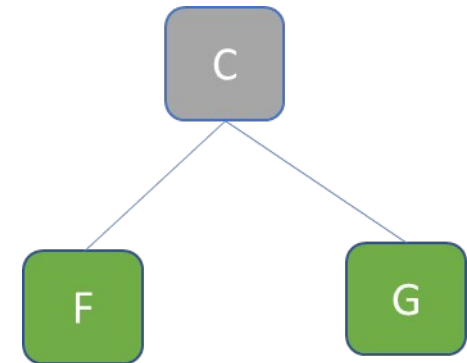
- Crear
- Insertar
- Eliminar
- Buscar
- Recorrido en profundidad
 - Preorden
 - Inorden
 - Postorden
- Recorrido en anchura

2. Árboles binarios de búsqueda (ABB). Introduccion

- Los árboles binarios no ordenados no tiene mucho uso.
 - Permiten la representación de información jerárquica de grado dos.
 - La búsqueda en un AB no ordenados no justifica su uso al de una estructura lineal (lista o arreglo).
- Los ABB son una solución eficiente para realizar búsquedas en colecciones ordenadas de datos
- También denominada BST (Binary Search Tree)

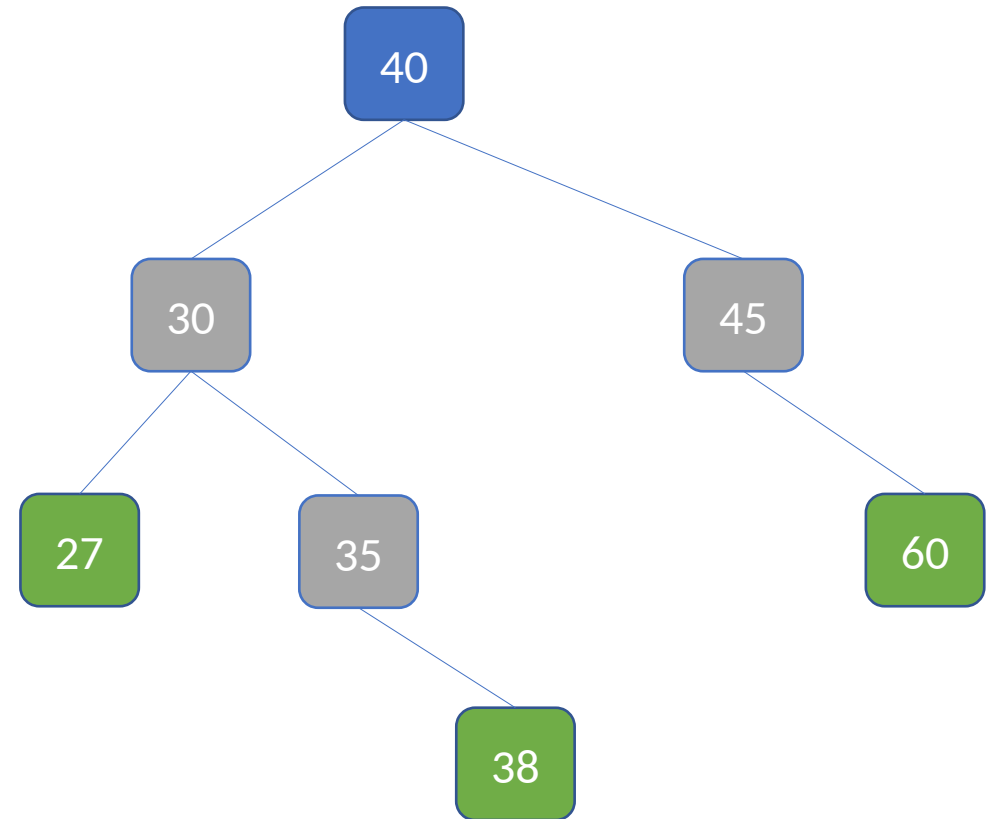
2. Árboles binarios de búsqueda (ABB)

- Un ABB es un árbol en donde todos sus nodos cumplen las siguientes condiciones:
 - Cada nodo esta asociada a una clave de ordenación.
 - Además para cada nodo, el valor de la clave de la raíz de su sub árbol izquierdo es menor que el valor de la clave del nodo.
 - El valor de la clave raíz del subárbol derecho es mayor que el valor de clave del nodo.
 - $G > C > F$ para los nodos del subárbol



3. ABB. definición

- Un ABB es un árbol en donde todos sus nodos cumplen las siguientes condiciones:
 - Cada nodo esta asociada a una clave de ordenación
 - La clave debe ser de tipo comparable.
 - La clave puede ser parte de una variable de tipo estructura como Alumno, Producto, Agenda, etc.
- Por ejemplo: Un ABB de alumnos del curso. La clave de cada nodo puede ser el código, el nombre , etc.



4. ABB. Ventajas y desventajas

- **Ventajas**

- Permiten almacenar una colección ordenada de datos o elementos de una forma mas eficiente al mejorar la complejidad la búsqueda.
- El numero de accesos al árbol es menor que en una lista.

- **Desventajas**

- Un ABB puede llegar a degenerar en una lista, si los elementos vienen ordenados ascendente o descendentemente.

5. ABB. Implementación

El árbol binario está formado de nodos

- Cada nodo consta de tres campos o partes:
 - Información: data o información del árbol.
 - Apuntador al hijo izquierdo.
 - Apuntador al hijo derecho.

REGISTRO NODO

NODO *hi

TD info

NODO *hd

FIN_REGISTRO

hi	info	hd
----	------	----

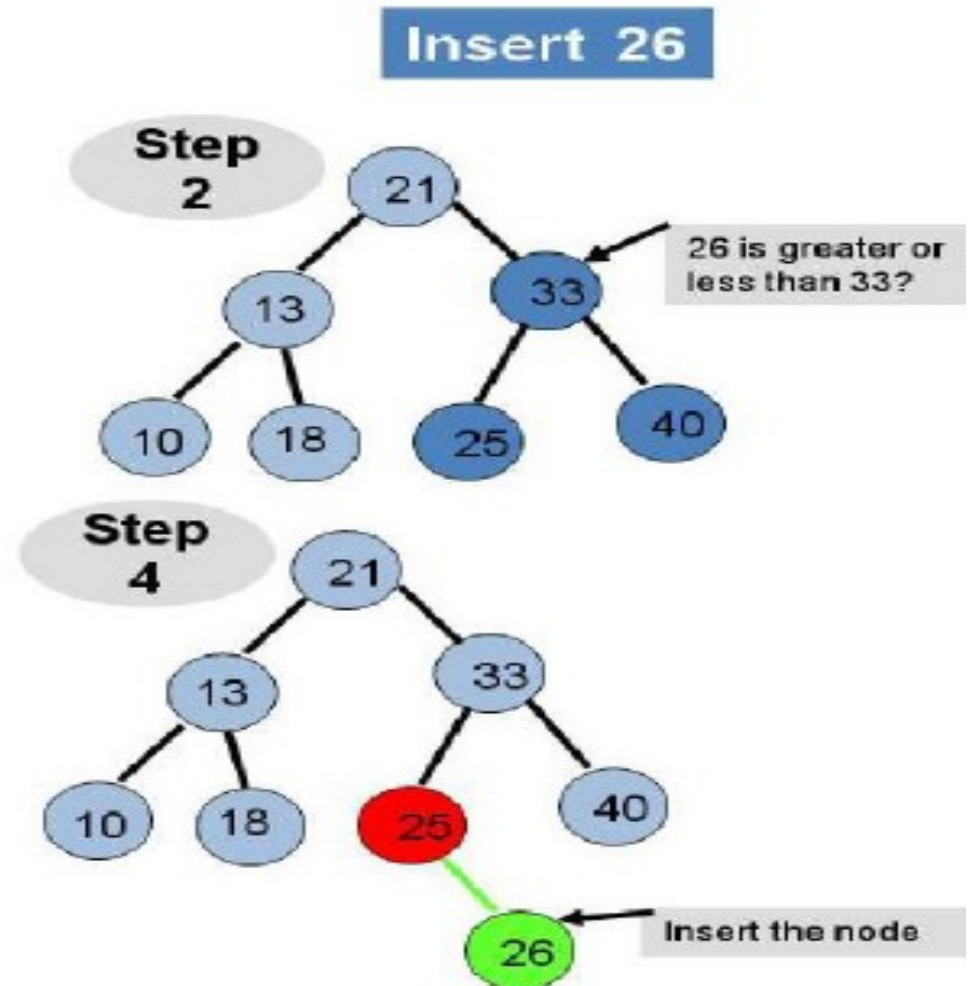
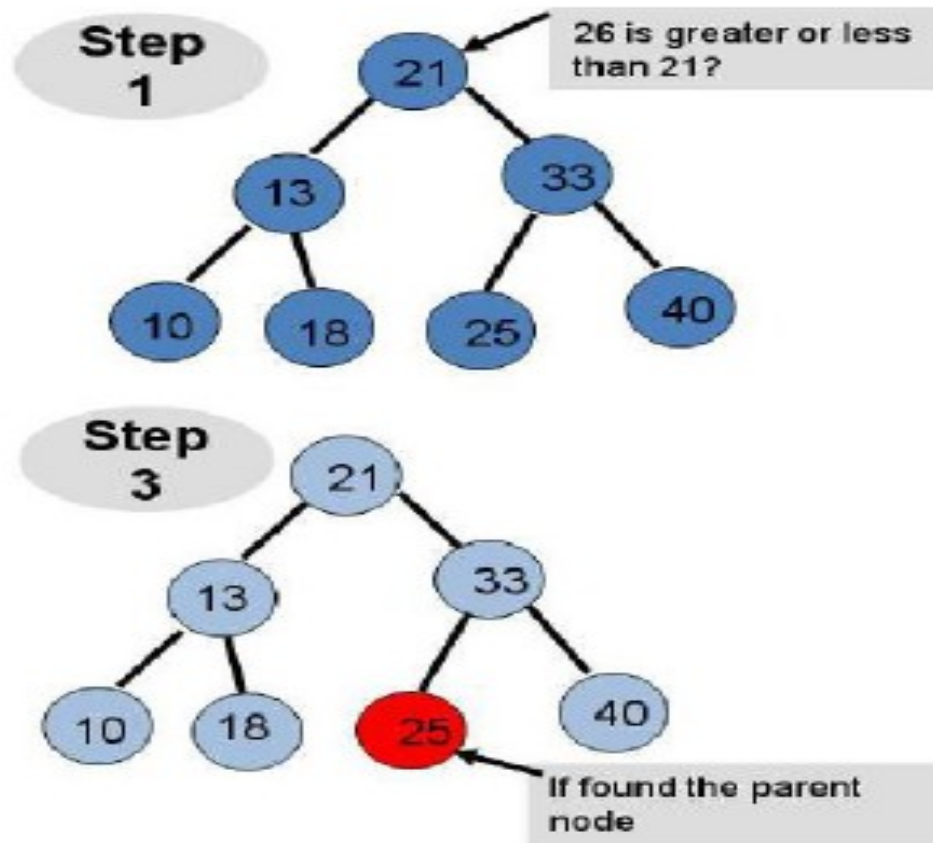
5. ABB: Operaciones

- Crear
- Insertar
- Eliminar
- Buscar
- Recorridos en profundidad
 - Preorden
 - Inorden
 - Postorden
- Recorrido en anchura

5. ABB: Inserción

- Algoritmo
 - Si el árbol esta vacío, el nodo se inserta como raíz, si no,
 - Se va recorriendo el árbol:
 - En cada nodo se decide si hay que insertar a la derecha o a la izquierda
 - Si el subarbol en que hay que insertar esta vacío, se inserta el nuevo nodo.
 - Si el subarbol en que hay que insertar no esta vacío hay que recorrerlo hasta encontrar el lugar que le corresponde al nodo en ese subarbol
- Implementación
 - El algoritmo puede ser recursivo
 - El algoritmo puede iterativo

5. ABB: Inserción, Ejemplo



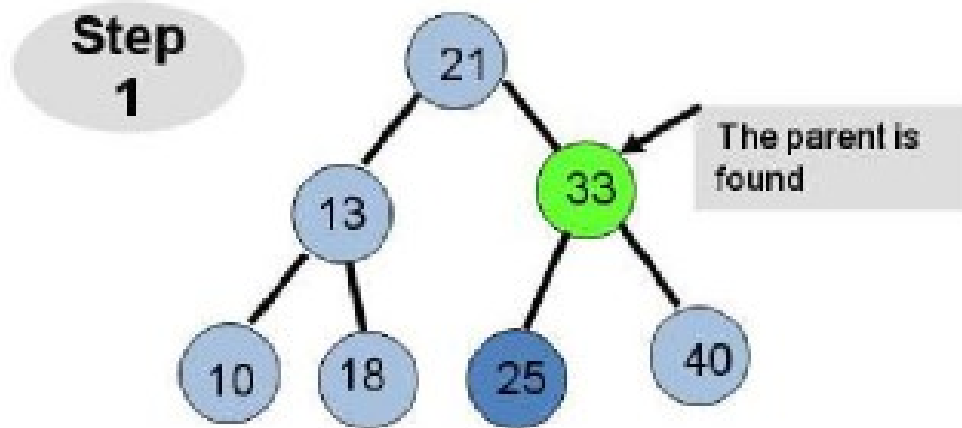
5. ABB: Búsqueda

- Algoritmo
 - Se va recorriendo el abb
 - Si el nodo actual no es el buscado, se decide la búsqueda por derecha o izquierda
 - El algoritmo se detiene al encontrar el nodo o encontrar el subárbol vacío.
- Implementación
 - El algoritmo puede ser recursivo
 - El algoritmo puede iterativo

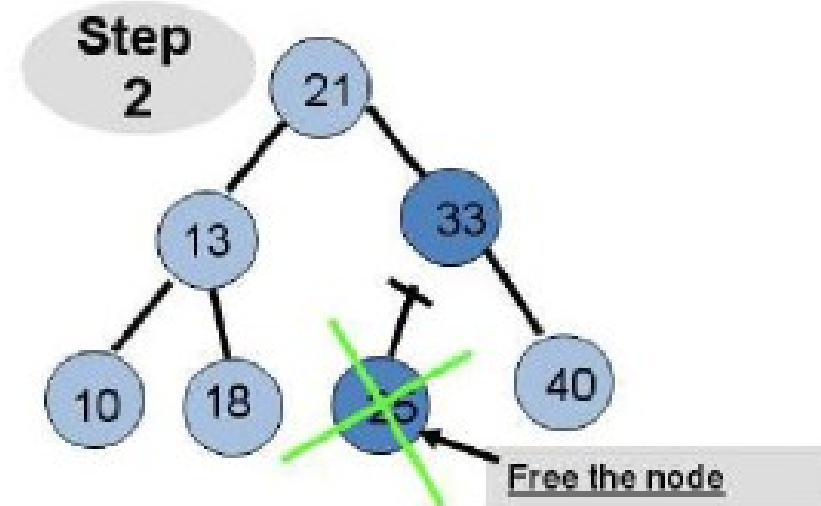
5. ABB: Eliminación

- Algoritmo
 - Si es un nodo hoja, basta con su padre haga referencia a vacío.
 - Si no es hoja, hay que sustituirlo por otro
 - El nodo a borrar solo tiene un hijo, sustituirlo por su hijo
 - El nodo a borrar tiene dos hijos, sustituirlo por:
 - El mayor de su subarbol izquierdo (predecesor) o
 - El menor de su subarbol derecho (sucesor)
 - En realidad no se sustituye un nodo por otro, sino que se reemplaza la clave y valor del nodo por la clave y valor del predecesor (o sucesor, segun la estrategia escogida). Ademàs sera necesario eliminar dicho nodo predecesor (o sucesor), que siempre sera un nodo hoja o un nodo con un unico hijo
 - Si el nodo a borrar es la raiz, hay que modificar la raiz, aplicando el caso que corresponda.

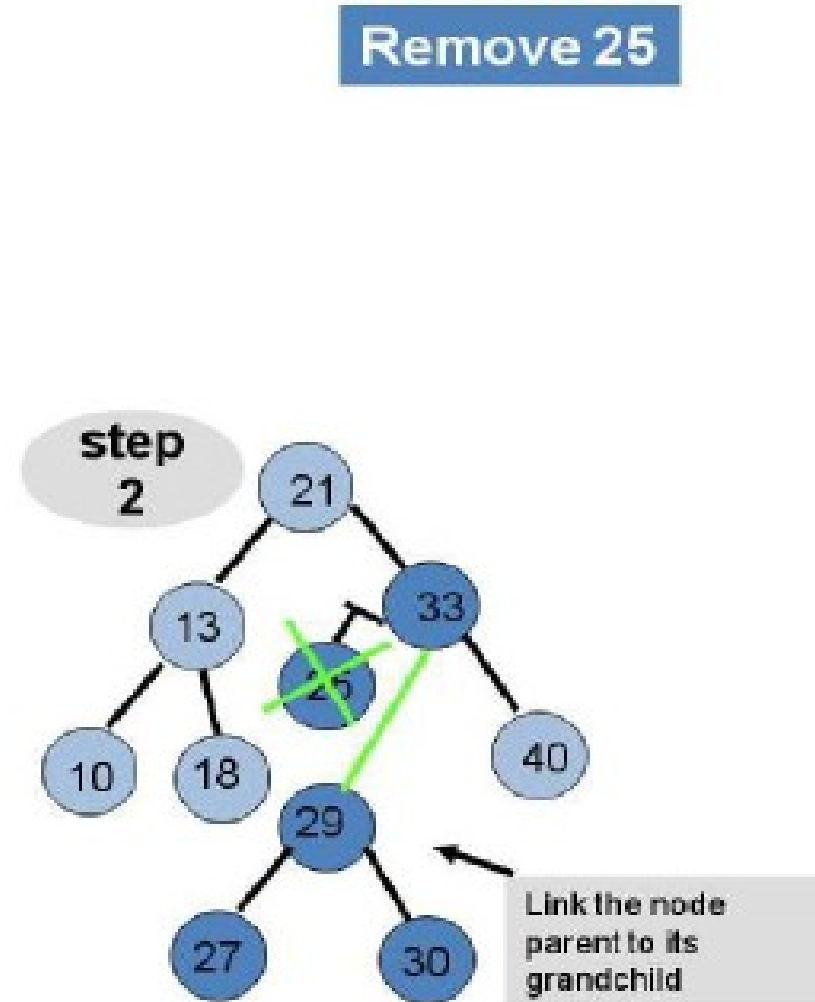
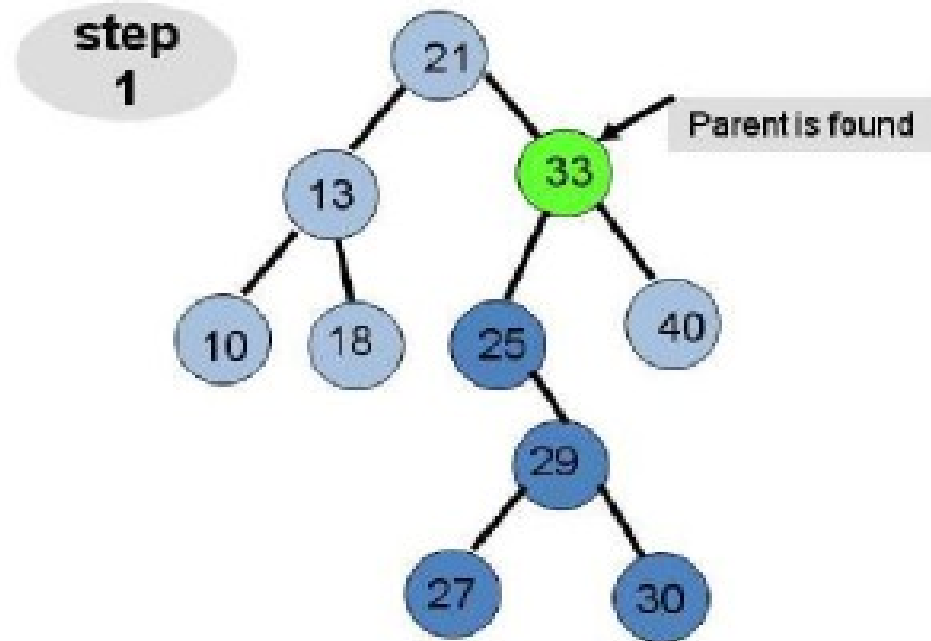
5. ABB Eliminación



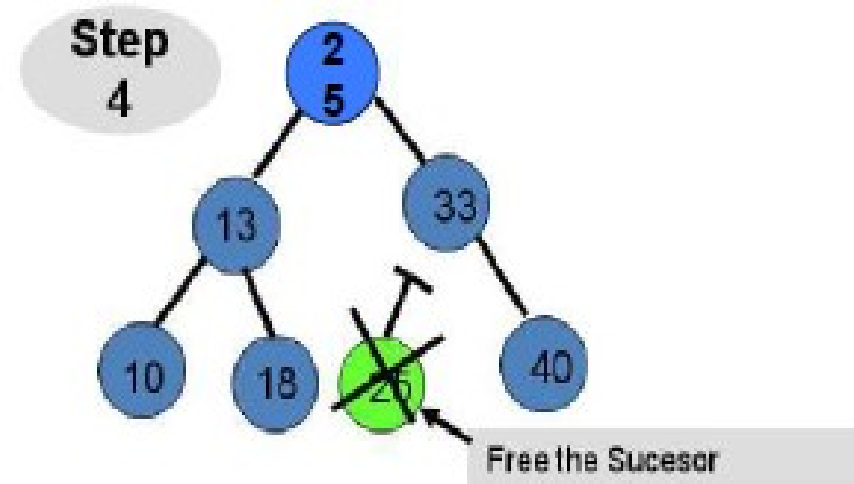
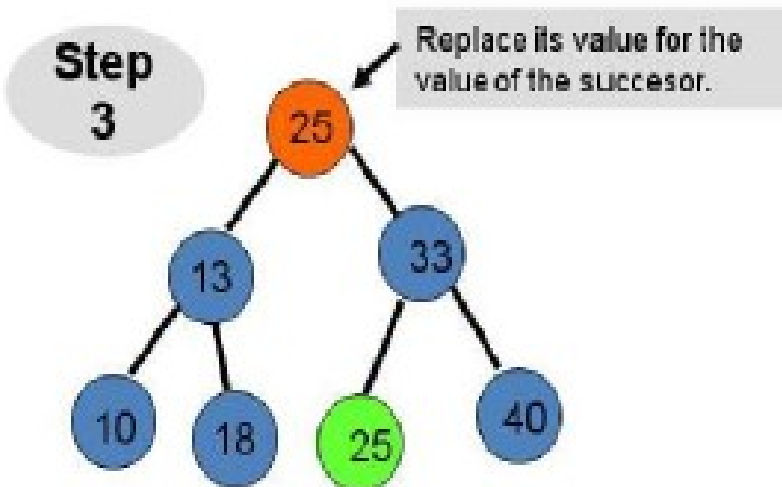
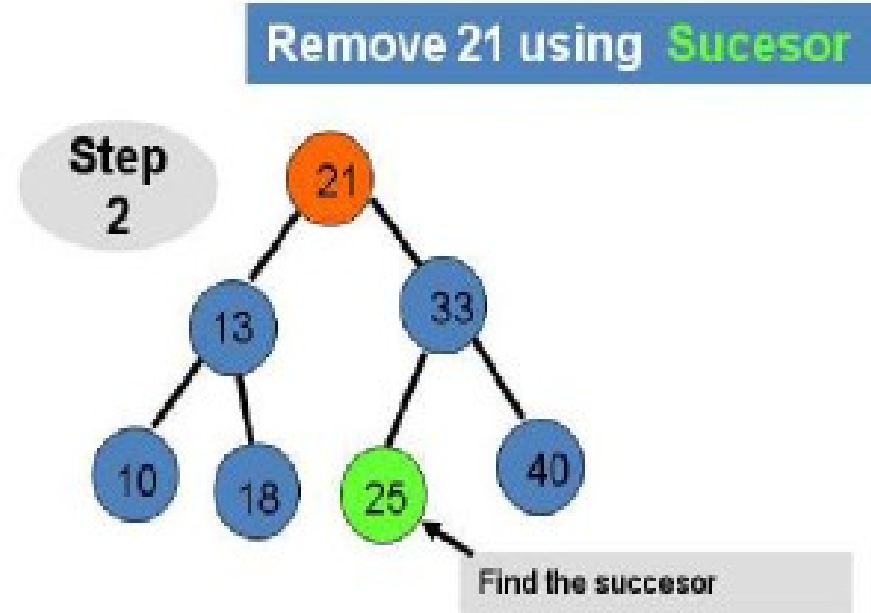
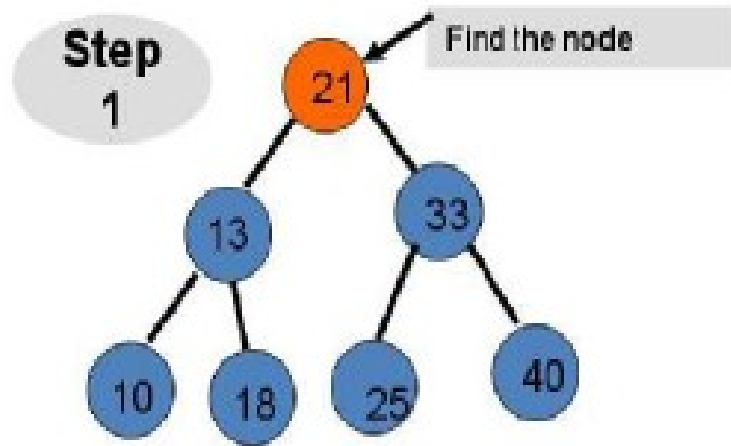
Remove 25



5. ABB: Eliminación



5. ABB: Eliminación



5. ABB: Recorrido en profundidad

ACCION preOrden (BNODO *n)

SI(n ≠ NULL)

ESCRIBIR (n.val)

PreOrden(n.hi)

PreOrden(n.hd)

FIN_SI

FIN_ACCION

ACCION InOrden (BNODO *n)

SI(n ≠ NULL)

InOrden(n.hi)

ESCRIBIR (n.val)

InOrden(n.hd)

FIN_SI

FIN_ACCION

ACCION PostOrden (BNODO *n)

SI(n ≠ NULL)

PostOrden(n.hi)

PostOrden(n.hd)

ESCRIBIR (n.val)

FIN_SI

FIN_ACCION

5. ABB: Recorrido en profundidad

Implementacion no recursiva:

ACCION preOrden (BNODO *n)

```
PILA p1,  
BNODO *t  
SI(n ≠ NULL)  
    Empilar(p1, n)  
    MIENTRAS (noVacia(p1))  
        Desempilar(p1, t)  
        ESCRIBIR(t)  
        SI(t.hd ≠ NULL)  
            Empilar(p1, t.hd)  
        FIN_SI  
        SI(t.hi ≠ NULL)  
            Empilar(p1, t.hi)  
        FIN_SI  
    FIN_MIENTRAS  
FIN_SI  
FIN_ACCION
```

ACCION inOrdenIteraNiveles (BNODO *n)

```
PILA p1.  
BNODO *t  
SI(N ≠ NULL)  
    t ← n.hi  
    empilar (p1, n)  
    MIENTRAS(t ≠ NULL O noVacia(p1))  
        SI(t ≠ NULL)  
            empilar(p1, t.hi)  
            t ← t.hi  
        SINO  
            t ← visorPila(p1)  
            depilar(p1)  
            ESCRIBIR(t.val)  
            t ← t.hd  
        FIN_SI  
    FIN_MIENTRAS  
FIN_SI  
FIN_ACCION
```

5. ABB: Recorrido en anchura

ACCION porNiveles (BNODO *n)

COLA c1.

BNODO t

SI(N ≠ NULL)

encolar (c1, n)

HACER

t decolar(c1))

ESCRIBIR(t.val)

SI(t.hi ≠ NULL)

encolar(C, t.hi)

FIN_SI

SI(t.hd ≠ NULL)

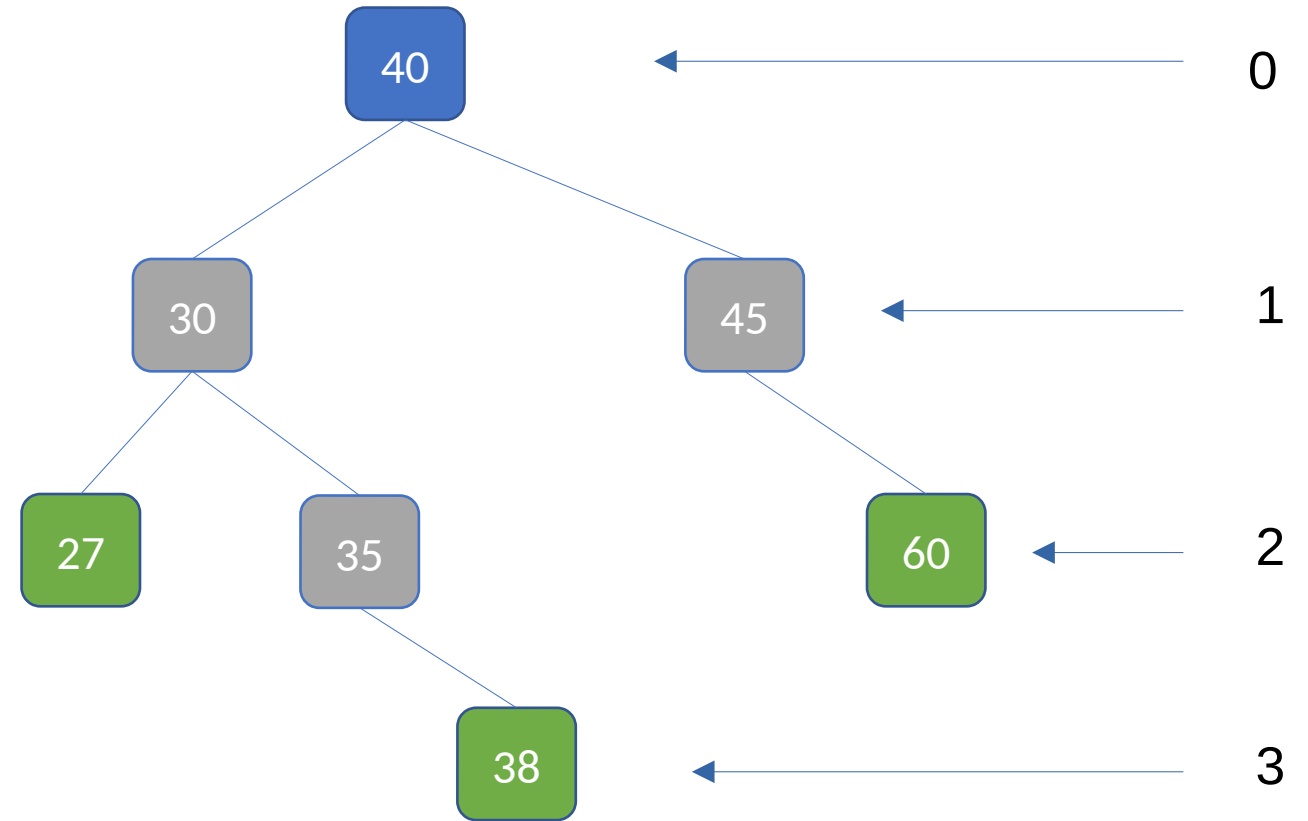
encolar(c1, t.hd)

FIN_SI

MIENTRAS (noVacia(c1))

FIN_SI

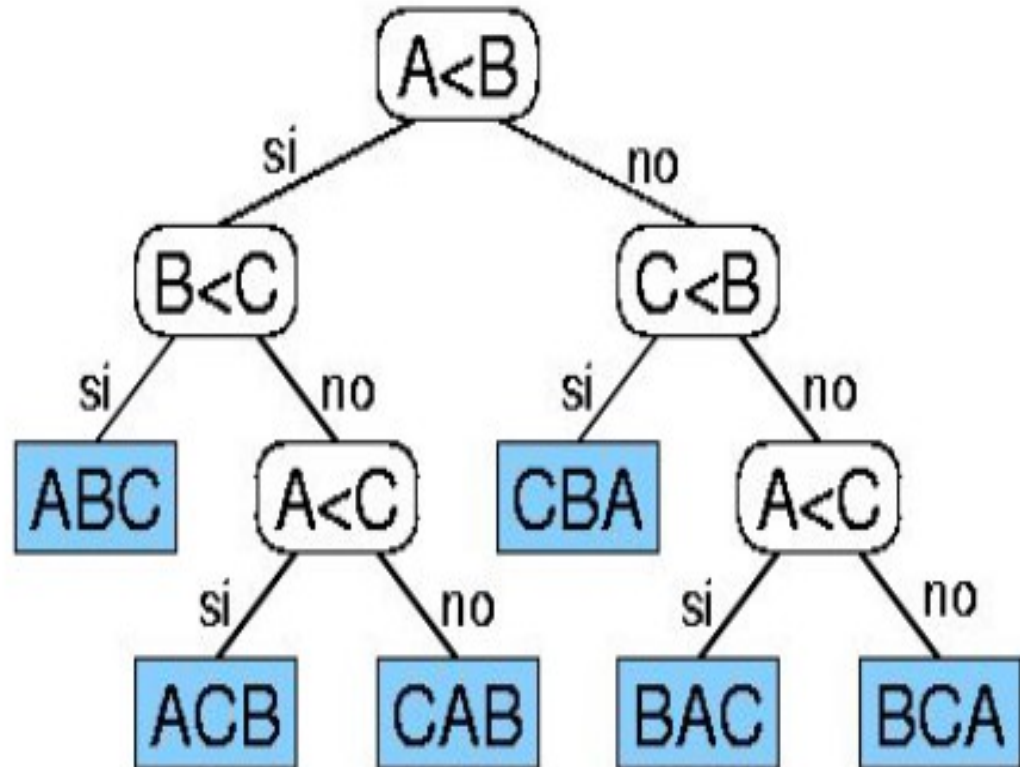
FIN_ACCION



Resultado: 40, 30, 45, 27, 35, 60, 38

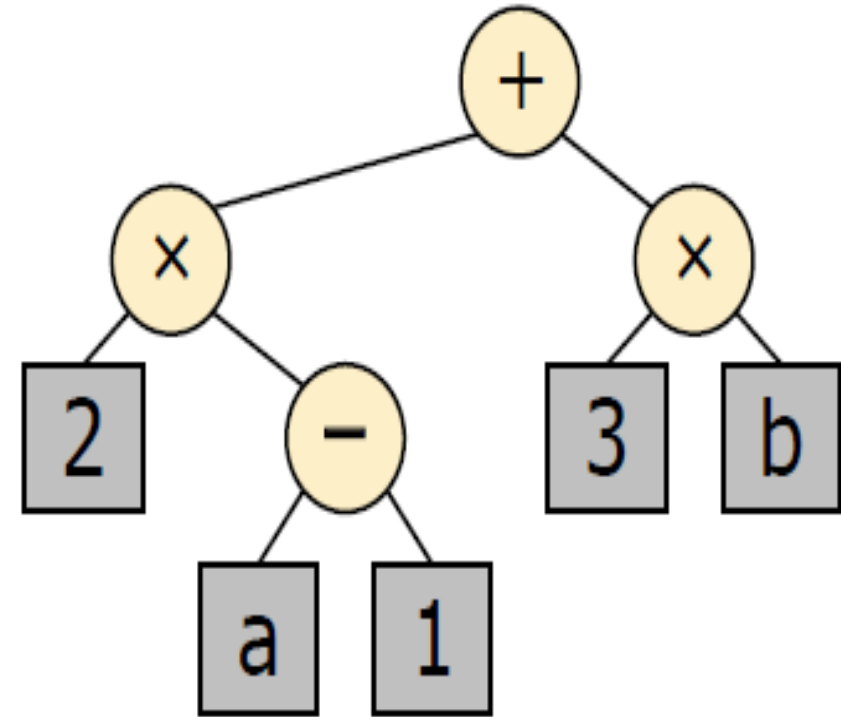
6. Árboles binarios: Aplicaciones

- Árboles de decisión.
 - Nodo interno: Preguntas con respuestas SI/NO
 - Nodos hoja: Decisiones



6. Árboles binarios: Aplicaciones

- Árboles para representar expresiones aritméticas.
 - Nodo interno: Operadores
 - Nodos hoja: operandos



7. Resumen

- Son arboles de grado dos.
- Son arboles eficientes para busqueda.
- Se degenera al recibir data ordenada
 - Se soluciona con arboles binarios AVL.

Referencias

- Joyanes L. y Zahonero I. (2004) Algoritmos y estructura de datos. Una perspectiva en C. Madrid España: McGraw-Hill.
- Cortez A. (2013) Algoritmica técnicas algorítmicas. Lima. Peru: CEPREDIN
- Joyanes L. y Zahonero I. (2008) Estructura de datos en Java. Madrid, España: McGraw-Hill.
- Cairo O y Guardati S. (2006) Estructura de datos. Mexico: McHraw-Hill.
- Villalobos J. (1996) Diseño y Manejo de Estructuras de Datos en C. Santa Fe de Bogota, Colombia: McGraw-Hill.
- Ficheros en C y C++ .[Ultima actualización 13.06.2017] [Accesado en Lima, 19.05.2020, 1820 horas], <http://c.conclase.net/ficheros/>
- Estructura de datos y Algoritmos. Universidad Carlos III de Madrid